

Machine Learning Algorithms

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



LMH
Lady Margaret Hall
University of Oxford

June 20, 2026

By the end of this session, you will be able to:

1. Navigate the ML Toolbox and distinguish between Linear, Distance-based, Kernel-based, and Tree-based models.
2. Understand Model Intuition and explain how these algorithms learn (e.g., Maximizing Margin in SVM, Greedy Splits in Trees).
3. Understand Ensembling and contrast Bagging (Random Forest) vs. Boosting (Gradient Boosting) and why they reduce error.
4. Understand the advantages and disadvantages of each algorithm.

1. Learning Outcomes
2. The Three Pillars of Learning
3. Supervised Learning
 1. Regression
4. Loss Functions
 1. Classification
5. Models
6. Machine Learning Models
 1. Linear Models
7. Linear Regression & Regularization
8. Logistic Regression & Softmax
 1. Distance-Based Models
9. k-Nearest Neighbors

Table of Contents (cont.)

- 1. Kernel-Based Models
- 10. Support Vector Machines
 - 1. Tree-Based Models
- 11. Decision Trees
- 12. Ensemble Methods
- 13. Conclusion
- 14. References
- 15. References

1. The Model (f_{θ})

Design Choice:
ML (Linear, Kernel, Trees)
& DL (MLP, CNN)

Depends on:
Data Type (Tabular, image)
& Complexity

2. Loss Function (J)

Design Choice:
MSE vs. Cross-Entropy

Depends on:
The Task
(Regression vs.
Classification)

3. Optimizer

Design Choice:
Gradient Descent, Adam

Depends on:
Stability & Con-
vergence Speed
(Covered later in
Deep Learning)

Today we focus on **1 & 2**.

1. The Model (f_θ)

Design Choice:
ML (Linear, Kernel, Trees)
& DL (MLP, CNN)

Depends on:
Data Type (Tabular, image)
& Complexity

2. Loss Function (J)

Design Choice:
MSE vs. Cross-Entropy

Depends on:
The Task
(Regression vs.
Classification)

3. Optimizer

Design Choice:
Gradient Descent, Adam

Depends on:
Stability & Con-
vergence Speed
(Covered later in
Deep Learning)

Today we focus on **1 & 2**.

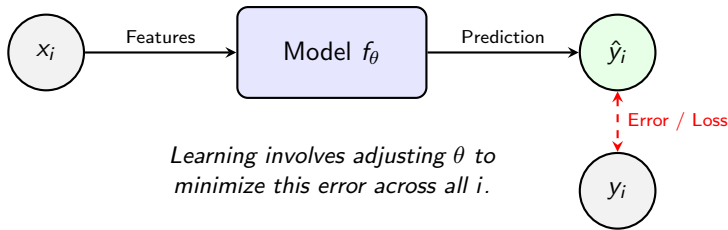
Let's start by discussing the loss functions first.

The Objective

Given a labeled dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, we aim to learn a mapping function f that generalizes to unseen data:

$$\hat{y} = f(x; \theta) \approx y$$

where θ represents the learnable model parameters.



Definition: The target y is **Continuous** ($y \in \mathbb{R}$).

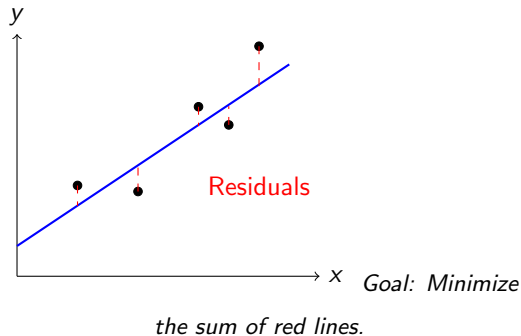
Examples:

- ▶ House Prices (\$300k, \$500k)
- ▶ Temperature (24.5°C)
- ▶ Stock Value

The Loss: Mean Squared Error

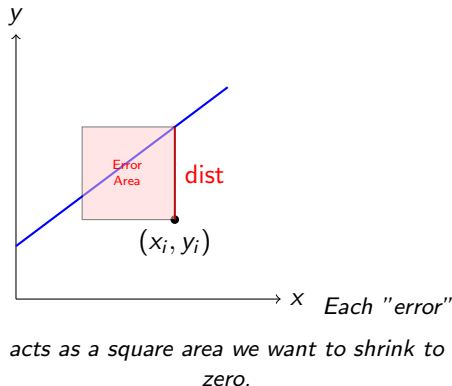
Measures the squared distance between points and the line.

$$\text{MSE} = \frac{1}{N} \sum (y - \hat{y})^2$$



The 3 Steps to MSE Loss:

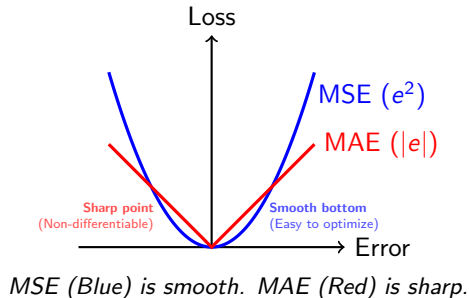
1. **Residuals** ($y - \hat{y}$): measures the raw distance. Points below the line give **negative** distances.
2. **The Square** $(\dots)^2$: Squaring ensures all distances are positive so they don't cancel out. It also heavily penalizes **outliers**.
3. **The Average** $(\frac{1}{N} \sum)$: We aggregate errors from all N samples to get a single performance score.



Note

If we use the **Absolute** difference $|y - \hat{y}|$ instead of the square, we get **MAE**. It is more robust to outliers, but **not differentiable** at 0.

Mathematically, you can still optimize functions with **finitely** many non-differentiable points using subgradients, though this adds some complexity.



Definition:

Target y is **discrete**: $y \in \{0, 1\}$ (e.g., spam/not spam)

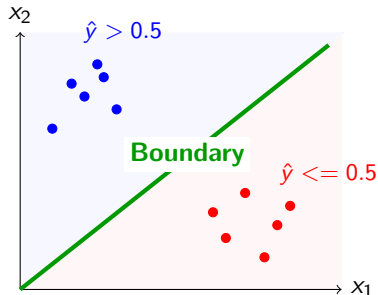
How It Works:

1. Model outputs **probability**: $\hat{y} \in [0, 1]$
2. Apply **threshold** (usually 0.5) to classify
3. If $\hat{y} \geq 0.5 \rightarrow$ Class 1, else Class 0

Example

$\hat{y} = 0.87 \xrightarrow{\text{threshold } 0.5} \text{Class 1 (Spam)}$

Decision Boundary Visualization



Goal: Find optimal boundary that maximizes class separation

Can We Use MSE Loss for Classification?

Problem with MSE

MSE measures **numeric distance**, but in classification, we care about **probability confidence**, not distance!

Problem with MSE

MSE measures **numeric distance**, but in classification, we care about **probability confidence**, not distance!

What We Really Want

A loss function that **heavily penalizes confident wrong predictions**

Example: Classifying an image as a dog with 98% confidence when it's actually a cat should incur massive penalty!

Can We Use MSE Loss for Classification?

Problem with MSE

MSE measures **numeric distance**, but in classification, we care about **probability confidence**, not distance!

What We Really Want

A loss function that **heavily penalizes confident wrong predictions**

Example: Classifying an image as a dog with 98% confidence when it's actually a cat should incur massive penalty!

Solution: The Logarithm!

The logarithm function has exactly the property we need:
it grows exponentially as confidence in the **wrong** class increases

Intuition (True Label $y = 1$):

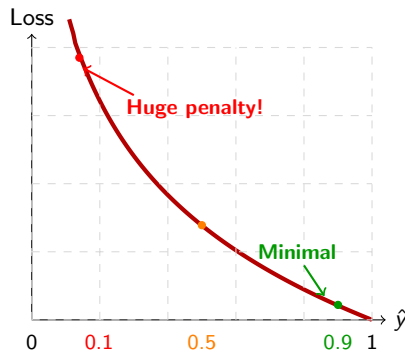
Model outputs probability $\hat{y}$ for class 1

Log Penalty = $-\log(\hat{y})$ behaves as:

- ▶ $\hat{y} = 0.99 \rightarrow \text{Loss} \approx 0.01$ ✓ **Great!**
- ▶ $\hat{y} = 0.50 \rightarrow \text{Loss} \approx 0.69$ **Uncertain**
- ▶ $\hat{y} = 0.01 \rightarrow \text{Loss} \approx 4.6$ ✗ **Disaster!**

The same concept applies when the true label is 0.

Loss vs. Predicted Probability (when true label = 1)



Curve explodes as $\hat{y} \rightarrow 0$
Forces model to be cautious!

Loss Function: Binary Cross-Entropy (Log Loss)

$$J(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

where: $y_i \in \{0, 1\}$ = true label, $\hat{y}_i \in (0, 1)$ = predicted probability, $\mathbf{w}$ = model weights.

- ▶ When $y_i = 1$: Loss = $-\log(\hat{y}_i)$ (penalizes low probability for class 1)
- ▶ When $y_i = 0$: Loss = $-\log(1 - \hat{y}_i)$ (penalizes high probability for class 1)
- ▶ Both terms encourage the model to predict probabilities close to the true labels.

But What If We Have More Than Two Classes?

Binary Classification:

- ▶ $y \in \{0, 1\}$
- ▶ Two classes only
- ▶ Example: Spam / Not Spam

Multiclass Classification:

- ▶ $y \in \{0, 1, 2, \dots, K - 1\}$
- ▶ K classes ($K > 2$)
- ▶ Example: Dog / Cat / Bird

Example: Image Classification

Input: Image $\rightarrow$ Output: $\hat{\mathbf{y}} = [0.1, 0.7, 0.2]$ for 3 classes
Class probabilities: Dog (10%), Cat (70%), Bird (20%)

Model now outputs a **probability distribution** over all K classes instead of a single probability

Loss Function: Cross-Entropy

Generalization of binary cross-entropy to K classes

$$J(\mathbf{w}) = -\frac{1}{n} \underbrace{\sum_{i=1}^n}_{\text{samples}} \underbrace{\sum_{k=1}^K}_{\text{classes}} y_{i,k} \log(\hat{y}_{i,k})$$

where $y_{i,k} \in \{0, 1\}$ is 1 if sample i belongs to class k , 0 otherwise

How It Works:

- ▶ Model outputs probabilities for all K classes: $\hat{\mathbf{y}}_i = [\hat{y}_{i,1}, \hat{y}_{i,2}, \dots, \hat{y}_{i,K}]$
- ▶ True label is one-hot encoded: $\mathbf{y}_i = [0, 1, 0, \dots, 0]$ if class 2
- ▶ Loss penalizes the probability assigned to the **true class**

Loss Function: Cross-Entropy

Generalization of binary cross-entropy to K classes

$$J(\mathbf{w}) = -\frac{1}{n} \underbrace{\sum_{i=1}^n}_{\text{samples}} \underbrace{\sum_{k=1}^K}_{\text{classes}} y_{i,k} \log(\hat{y}_{i,k})$$

where $y_{i,k} \in \{0, 1\}$ is 1 if sample i belongs to class k , 0 otherwise

Example

True class: Cat (class 2), Prediction: $[0.1, 0.7, 0.2]$

Loss = $-\log(0.7) \approx 0.36$ (only the true class matters!)

1. The Model (f_θ)

Design Choice:
ML (Linear, Kernel, Trees)
& DL (MLP, CNN)

Depends on:
Data Type (Tabular, image)
& Complexity

2. Loss Function (J)

Design Choice:
MSE vs. Cross-Entropy

Depends on:
The Task
(Regression vs.
Classification)

3. Optimizer

Design Choice:
Gradient Descent, Adam

Depends on:
Resources & Scale
(Covered later in
Deep Learning)

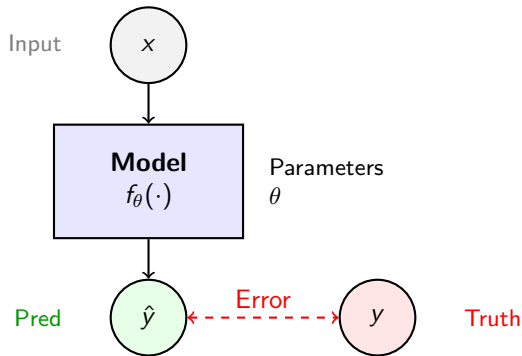
We have covered the **Loss Functions**.

Now, let's move on to the **Models** themselves (f_θ).

Definition

A **Model** (f_{θ}) is a simplified mathematical representation of reality. It maps inputs (x) to outputs ($\hat{y}$) using learned parameters (θ).

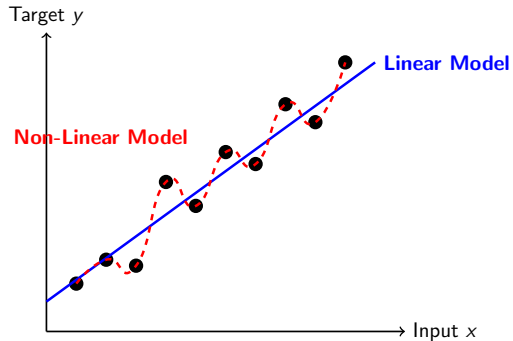
The Goal: Find the "best" function f that minimizes the error between prediction $\hat{y}$ and reality y .



The relationship between inputs $\mathbf{x}$ and target y varies wildly. Choosing the correct model family matters to avoid **Underfitting** (too simple) or **Overfitting** (too complex).

- ▶ **Simple:** Linear.
(e.g., *House Size* $\rightarrow$ *Price*)
- ▶ **Complex:** Non-linear.
(e.g., *Image Pixels* $\rightarrow$ "Cat")

Models Complexity (cont.)



1. Machine Learning (ML)

Algorithms that parse data, learn patterns from it, and apply learned decisions.

Characteristics:

- ▶ Best for **Structured Data** (Tables).
- ▶ Require **Feature Engineering**.
- ▶ Lightweight & Interpretable (mostly).

Examples: *Linear Reg, SVM, Trees*

2. Deep Learning (DL)

A subset of Machine Learning based on artificial neural networks.

Characteristics:

- ▶ Best for **Unstructured Data** (Images, Text, Audio).
- ▶ **Automated** Feature Extraction.
- ▶ Data & Compute Hungry.

Examples: *MLPs, CNNs, Transformers*

1. Machine Learning (ML)

Algorithms that parse data, learn patterns from it, and apply learned decisions.

Characteristics:

- ▶ Best for **Structured Data** (Tables).
- ▶ Require **Feature Engineering**.
- ▶ Lightweight & Interpretable (mostly).

Examples: *Linear Reg, SVM, Trees*

**Let's start with
Machine Learning first.**

Machine Learning Models (f_{θ})

1. Linear Models

Linear Regression
Logistic Regression

2. Distance-Based

KNN
(Non-Parametric)

3. Kernel-Based

SVM

4. Tree-Based

Decision Trees
Random Forest
Gradient Boosting

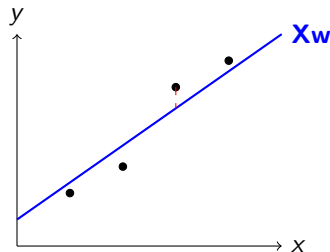
↑ We Start Here!

We start with the simplest assumption:
The target y is a **linear combination** of the input features.

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_dx_d$$

The Components:

- ▶ $\hat{y}$: predicted value
- ▶ $x_1, \dots, x_d$: the input features.
- ▶ $w_1, \dots, w_d$: the weights. How much each feature matters.
- ▶ w_0 : the bias (Intercept).



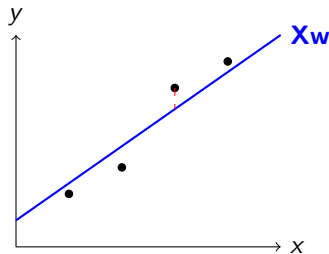
We start with the simplest assumption:
The target y is a **linear combination** of the input features.

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d$$

The Components:

- ▶ $\hat{y}$: predicted value
- ▶ $x_1, \dots, x_d$: the input features.
- ▶ $w_1, \dots, w_d$: the weights. How much each feature matters.
- ▶ w_0 : the bias (Intercept).

Writing long sums ($\sum$) is tedious. We use Linear Algebra to represent Linear Regression in matrices notation.



Step 1: The Bias Trick: We add a "dummy" feature $x_0 = 1$ to the input so we have each weight multiplied by a x feature.

$$\hat{y} = w_0(1) + w_1x_1 + \dots + w_dx_d$$

Step 2: Vector Notation (For one sample): We rewrite this equation in a vector dot product form:

$$\hat{y} = \begin{bmatrix} w_0 & w_1 & \dots & w_d \end{bmatrix} \cdot \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_d \end{bmatrix} = \mathbf{w}^T \mathbf{x}$$

Step 3: Matrix Notation (For all data): For N samples, prediction becomes a simple Matrix-Vector multiplication:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$$

Now we have the equation. But how do we choose the best $\mathbf{w}$?

Now we have the equation. But how do we choose the best $\mathbf{w}$?
We need the loss and the optimizer!

Now we have the equation. But how do we choose the best $\mathbf{w}$?
We need the loss and the optimizer!

Let's wrap our model in the Mean Squared Error:

$$J(\mathbf{w}) = \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

Now we have the equation. But how do we choose the best $\mathbf{w}$?
We need the loss and the optimizer!

Let's wrap our model in the Mean Squared Error:

$$J(\mathbf{w}) = \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

Now, how do we find the $\mathbf{w}$ that minimizes this $J(\mathbf{w})$?

Now we have the equation. But how do we choose the best $\mathbf{w}$?
We need the loss and the optimizer!

Let's wrap our model in the Mean Squared Error:

$$J(\mathbf{w}) = \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

Now, how do we find the $\mathbf{w}$ that minimizes this $J(\mathbf{w})$?

Option A: Analytical
(Solve for zero gradient)

OR

Option B: Iterative
(Gradient Descent)

Method A: The Normal Equation

For linear models, MSE is **convex**. **Q:** What does that tell you?

Method A: The Normal Equation

For linear models, MSE is **convex**. **Q:** What does that tell you? *Guaranteed Global Minimum!*

For linear models, MSE is **convex**. **Q:** What does that tell you? *Guaranteed Global Minimum!*

Step 1: The Gradient Derived via Chain Rule:

$$\nabla J(\mathbf{w}) = \frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

For linear models, MSE is **convex**. **Q:** What does that tell you? *Guaranteed Global Minimum!*.

Step 1: The Gradient Derived via Chain Rule:

$$\nabla J(\mathbf{w}) = \frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Step 2: Set to Zero

$$\mathbf{X}^T \mathbf{X}\mathbf{w} - \mathbf{X}^T \mathbf{y} = 0$$

$$\mathbf{X}^T \mathbf{X}\mathbf{w} = \mathbf{X}^T \mathbf{y}$$

Method A: The Normal Equation

For linear models, MSE is **convex**. **Q:** What does that tell you? *Guaranteed Global Minimum!*

Step 1: The Gradient Derived via Chain Rule:

$$\nabla J(\mathbf{w}) = \frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Step 2: Set to Zero

$$\begin{aligned}\mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{y} &= 0 \\ \mathbf{X}^T \mathbf{X} \mathbf{w} &= \mathbf{X}^T \mathbf{y}\end{aligned}$$

Step 3: Solve for $\mathbf{w}$ Multiply both sides by inverse $(\mathbf{X}^T \mathbf{X})^{-1}$:

The Normal Equation

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Cons: Inverting large matrices is very slow ($O(d^3)$). Doesn't work if X is singular.

Instead of jumping to the solution, we take small steps.

Step 1: Calculate Gradient Derived via Chain Rule:

$$\nabla J(\mathbf{w}) = \frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Step 2: Update Rule Move opposite to the gradient to reduce error.

Iterative Update

$$\mathbf{w}_{new} = \mathbf{w}_{old} - \alpha \nabla J$$

α : Learning Rate (Step size)

Pros: Very efficient for large datasets.

Imagine we train two linear models on the same data. Both achieve **Training MSE = 0.10**. Which one is better?

Model A

$$y = 0.5x_1 + 0.3x_2$$

Model B

$$y = 500.5x_1 - 429.7x_2$$

Imagine we train two linear models on the same data. Both achieve **Training MSE = 0.10**. Which one is better?

Model A (Small Weights)

$$y = 0.5x_1 + 0.3x_2$$

Val MSE: 0.12

Stable. Why?

→ Small changes in x cause **Small** changes in y .

Model B (Huge Weights)

$$y = 500.5x_1 - 429.7x_2$$

Val MSE: 15.0 (!!!)

Unstable. Why?

→ Small changes in x cause **Massive** changes in y .

Imagine we train two linear models on the same data. Both achieve **Training MSE = 0.10**. Which one is better?

Model A (Small Weights)

$$y = 0.5x_1 + 0.3x_2$$

Val MSE: 0.12

Stable. Why?

→ Small changes in x cause **Small** changes in y .

Model B (Huge Weights)

$$y = 500.5x_1 - 429.7x_2$$

Val MSE: 15.0 (!!!)

Unstable. Why?

→ Small changes in x cause **Massive** changes in y .

Observation: Large weights are often a symptom of **Overfitting**. The model is trying too hard to wiggle through every training point.

The Fix: Constraining the Weights

We must force the model to keep weights small. We do this by adding a **Constraint** (Penalty) to the loss function.

Minimize Error

Minimize Weights

$$J_{final} = \overbrace{J_{MSE}}^{\text{Fit the Data}} + \overbrace{\lambda \cdot R(\mathbf{w})}^{\text{Keep Weights Small}}$$

What is λ (Lambda)?

Regularization Strength (Hyperparameter):

- ▶ **High** λ : Huge penalty $\rightarrow$ Tiny weights (Model becomes simple) $\rightarrow$ Increases underfitting.
- ▶ **Low** λ : Small penalty $\rightarrow$ Weights can grow (Model fits data closely) $\rightarrow$ Increases overfitting.
- ▶ $\lambda = 0$: Standard Linear Regression (No penalty).

Let's set $\lambda = 1$ and Penalty $R(\mathbf{w}) = w^2$. Compare two models that have similar error, but different weights.

Model A (Overfit)

Weight $w = 100$

MSE Error = 0 (Perfect fit)

Total Loss:

$$J = 0 + 1 \cdot (100)^2$$

$$J = 10,000$$

HUGE LOSS!

Model B (Balanced)

Weight $w = 2$

MSE Error = 5 (Some error)

Total Loss:

$$J = 5 + 1 \cdot (2)^2$$

$$J = 9$$

WINNER!

How do we define the penalty $R(\mathbf{w})$? We have two main flavors:

- ▶ **Ridge (L2):** $R(\mathbf{w}) = \sum w^2$ (Squared)
- ▶ **Lasso (L1):** $R(\mathbf{w}) = \sum |w|$ (Absolute)

We penalize the **Squared Magnitude** of the weights.

Objective Function

$$J(\mathbf{w}) = \text{MSE} + \lambda \sum w^2$$

Effect: "Weight Decay"

- ▶ Shrinks weights smoothly toward 0 (stability).
- ▶ Usually does not make weights exactly 0 (not sparse).
- ▶ Works well with correlated features.

We penalize the **Absolute Value** of the weights.

Objective Function

$$J(\mathbf{w}) = \text{MSE} + \lambda \sum |w|$$

Effect: "Sparsity"

- ▶ Encourages **sparsity**: many weights become exactly 0.
- ▶ Acts as **feature selection**.
- ▶ Good when we believe **only a few features matter**.

How do we find the optimal weights w ?

1. Ridge (L2): Has a Closed-Form Solution

Ridge Normal Equation

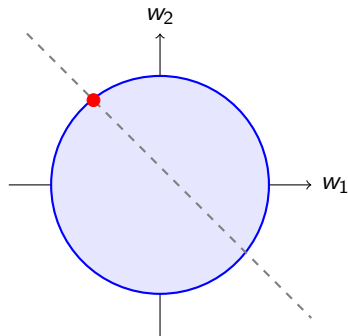
$$\mathbf{w} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$$

**Note: Adding $\lambda \mathbf{I}$ ensures the matrix is always invertible.*

2. Lasso (L1): No Closed-Form Solution

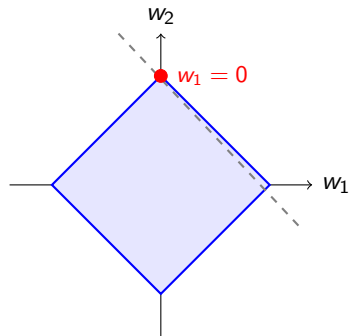
- ▶ Because the absolute value function $|w|$ is not differentiable at 0, we cannot derive a simple formula.
- ▶ We must use **Iterative Algorithms** to find the minimum.

L1 vs L2: Geometric Intuition



Loss Contour + L2 Constraint

Ridge: Smooth solution, rarely hits axis
 $w_1 = -1.5, w_2 = 2.0$



Loss Contour + L1 Constraint

Lasso: Sharp corners promote sparsity
 $w_1 = 0, w_2 = 2.5$

The Challenge: We want to classify data (e.g., Spam vs Not Spam).

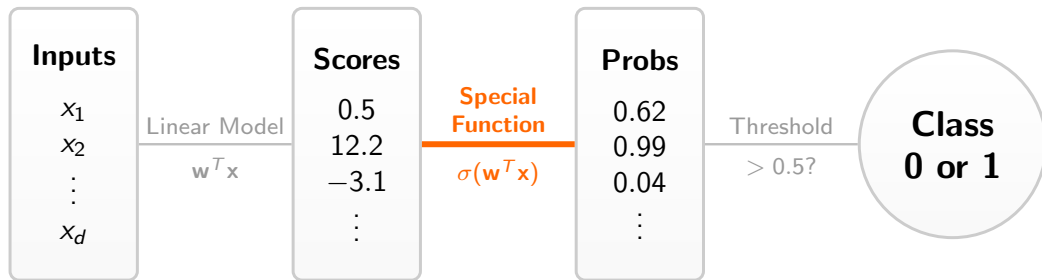
However, our Linear Model $f(x) = \mathbf{w}^T \mathbf{x}$ outputs numbers in $(-\infty, \infty)$.

Problem

If model outputs 500 or -20 , how do we interpret that as a probability?

We need a mechanism to map these raw "scores" into a valid probability range $[0, 1]$.

The Classification Pipeline



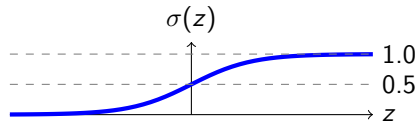
The "Special Function" we need is the **Sigmoid** (Logistic Function).

Sigmoid Function

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Properties:

- ▶ Squashes any z into $(0, 1)$.
- ▶ $z = 0 \implies \hat{y} = 0.5$.
- ▶ Differentiable everywhere (Smooth).



To train, we need to minimize the Binary Cross-Entropy Loss:

$$J = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Let's compute $\frac{\partial J}{\partial w}$ using the Chain Rule:

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

The Three Components:

1. **Loss Derivative:** $\frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$

2. **Sigmoid Derivative:** $\hat{y}(1 - \hat{y})$

3. **Linear Derivative:** x

(Can you derive this?)

Now, multiply them together:

$$\frac{\partial J}{\partial w} = \left(\frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \right) \cdot \hat{y}(1 - \hat{y}) \cdot x$$

The complicated Sigmoid derivative perfectly cancels the denominator of the Log Loss!

Final Gradient Update

$$\nabla J(\mathbf{w}) = \frac{1}{N} \mathbf{X}^T (\hat{\mathbf{y}} - \mathbf{y})$$

Observation

This formula is **identical** to Linear Regression (ignore constants)! The only difference is how we calculate $\hat{y}$ (Sigmoid vs. Identity).

Generalization: Multiclass (Softmax)

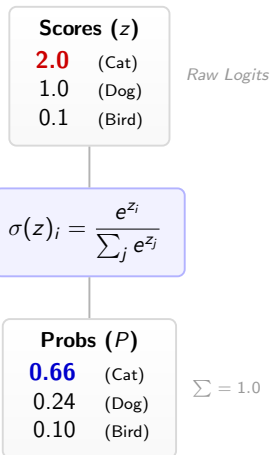
What if we have $K > 2$ classes? (e.g., Cat, Dog, Bird). We generalize Sigmoid to **Softmax**.

Softmax Function

$$P(y = k) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

- ▶ **Exponent (e^z):** Forces all scores to be positive.
- ▶ **Sum ($\sum$):** Normalizes so all probabilities sum to 1.0.
- ▶ **Loss:** Cross-Entropy.

$$J = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{i,k} \log(\hat{y}_{i,k})$$



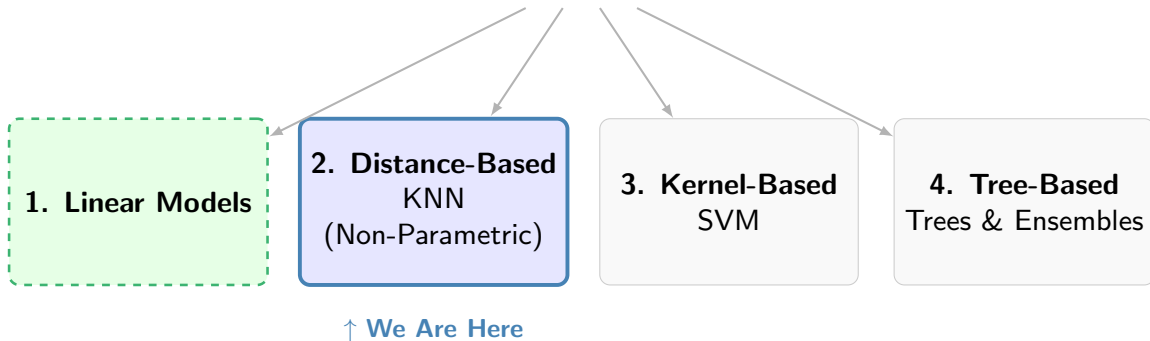
Advantages

- ▶ **Interpretable:** The weights w directly tell us feature importance.
- ▶ **Fast & Efficient:** Training is simple matrix math. Inference is just a dot product.

Disadvantages

- ▶ **High Bias (Underfitting):** Cannot model complex, non-linear relationships.
- ▶ **Sensitive to Outliers.** Especially when using MSE.
- ▶ **Scaling Required:** Sensitive to feature magnitude (Requires Standardization).

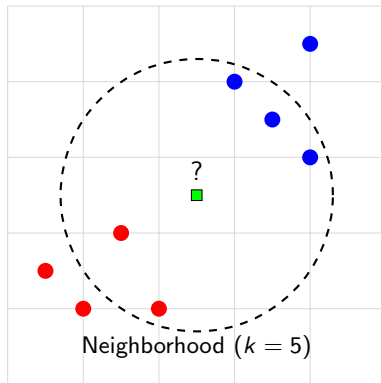
Machine Learning Models (f_θ)



Linear models try to learn a global formula ($\mathbf{w}^T \mathbf{x}$). **k-NN** is different: it is a **Lazy Learner**. It memorizes the training data and makes predictions based on local similarity. So, it has **no parameters**.

The Algorithm:

1. Choose a distance metric $d(\cdot, \cdot)$ and an integer k .
2. For a new point x_{new} , find the k training points closest to it.
3. **Classification:** Take the majority vote.
4. **Regression:** Take the average.



The performance of k-NN depends heavily on how we define "closeness."

1. Euclidean Distance (L_2 Norm): Most common

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^d (x_i - y_i)^2} = \|\mathbf{x} - \mathbf{y}\|_2$$

2. Manhattan Distance (L_1 Norm): Good for high dimensions

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^d |x_i - y_i| = \|\mathbf{x} - \mathbf{y}\|_1$$

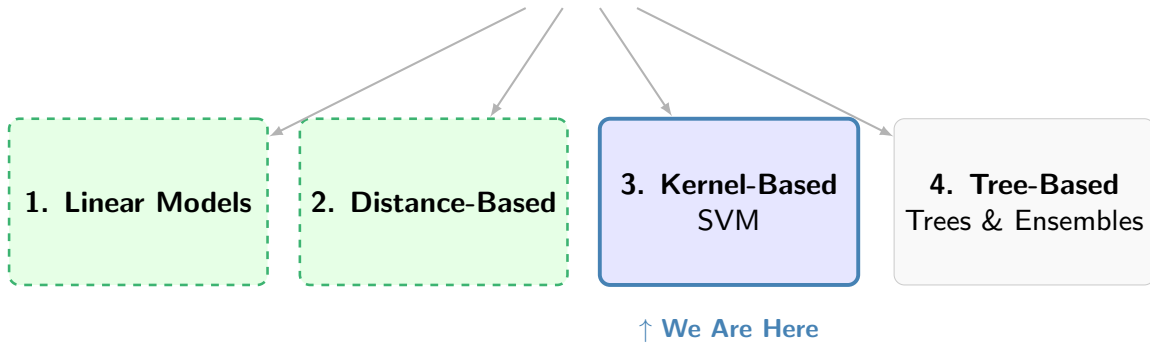
Advantages

- ▶ **Simple:** Easy to understand and implement.
- ▶ **No Training:** Training time is effectively zero ($O(1)$).

Disadvantages

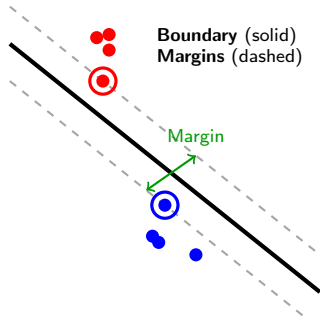
- ▶ **Slow Inference:** Prediction is $O(N)$ (must scan all data).
- ▶ **Memory Intensive:** Must store the entire dataset.
- ▶ **Scaling Required:** Sensitive to feature magnitude (Requires Standardization).
- ▶ **Curse of Dimensionality:** Performance degrades noticeably in high dimensions.

Machine Learning Models (f_{θ})



"Success isn't about barely crossing the line. It's about having some room to spare."

Logistic Regression learns a boundary that fits probabilities. SVM chooses the boundary with the **largest safety gap (Margin)**.



- ▶ **Hyperplane:** boundary ($\mathbf{w}^T \mathbf{x} + b = 0$).
- ▶ **Margin:** "safety gap" to the closest points.
- ▶ **Support Vectors:** closest points that define the boundary.

Bigger margin $\Rightarrow$ more robust to noise.

We established that we want the **widest possible margin**. Mathematically, maximizing width $\iff$ minimizing weights $\|\mathbf{w}\|$.

The "Hard Margin" Objective

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

Subject to: $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$

What does this mean?

- ▶ **Minimize Objective:** Keep the weights small.
- ▶ **Subject to Constraint:** Strictly forbid ANY data point from touching the margin (i.e., being misclassified).

**Note: This assumes data is perfectly linearly separable (Hard Margin).*

Real data is noisy (never perfectly linearly separable). A "Hard Margin" would crash if just one outlier existed in the data.

Solution: We relax the rules. We allow some "violations" (ξ_i) but penalize them using a hyperparameter **C**.

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + \mathbf{C} \sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

C weights the *total* violation against the margin term, so it shapes *all* the weights **w** globally — not just the outlier points.

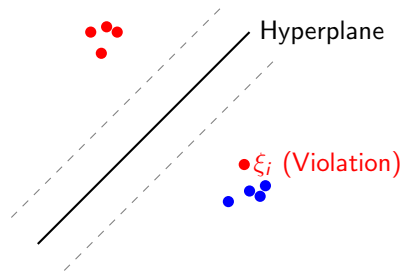
The Hyperparameter C

Large C (Strict)

- ▶ Penalizes errors heavily.
- ▶ Result: Narrow margin, risk of overfitting.

Small C (More Violations)

- ▶ Ignores small errors.
- ▶ Result: Wider margin, better generalization.



"Ideally, separate perfectly. If not, minimize the sum of violations."

Problem: Standard SVM finds a linear boundary. What if data is not linearly separable (e.g., a donut)?

Problem: Standard SVM finds a linear boundary. What if data is not linearly separable (e.g., a donut)?

The Solution (Feature Engineering): We map inputs to a higher-dimensional space where they *become* separable.

$$\mathbf{x} \rightarrow \phi(\mathbf{x})$$

(e.g., adding x^2 as a new feature).

Problem: Standard SVM finds a linear boundary. What if data is not linearly separable (e.g., a donut)?

The Solution (Feature Engineering): We map inputs to a higher-dimensional space where they *become* separable.

$$\mathbf{x} \rightarrow \phi(\mathbf{x})$$

(e.g., adding x^2 as a new feature).

Issue: Computing $\phi(\mathbf{x})$ explicitly is computationally expensive.

Problem: Standard SVM finds a linear boundary. What if data is not linearly separable (e.g., a donut)?

The Solution (Feature Engineering): We map inputs to a higher-dimensional space where they *become* separable.

$$\mathbf{x} \rightarrow \phi(\mathbf{x})$$

(e.g., adding x^2 as a new feature).

Issue: Computing $\phi(\mathbf{x})$ explicitly is computationally expensive.

The "Trick" (Implicit Computation): We use special type of functions called **Kernel Functions** which compute the high-dim **similarity directly** without explicitly building them!

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

This is called the **Kernel Trick**.

The choice of Kernel tells the SVM "how to measure similarity."

► 1. Linear Kernel

(Good for Linear Problems)

- $K(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T \mathbf{y}$
- **What it finds:** Straight lines/planes.

► 2. Polynomial Kernel

- $K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + c)^d$
- **What it finds:** Curved boundaries and feature interactions.

► 3. Radial Basis Function (RBF)

(The Default)

- $K(\mathbf{x}, \mathbf{y}) = \exp(-\gamma \|\mathbf{x} - \mathbf{y}\|^2)$
- **What it finds:** "Local" similarity. Points are similar only if they are close in distance.

Rule of Thumb: Start with **RBF** if you don't know the structure.

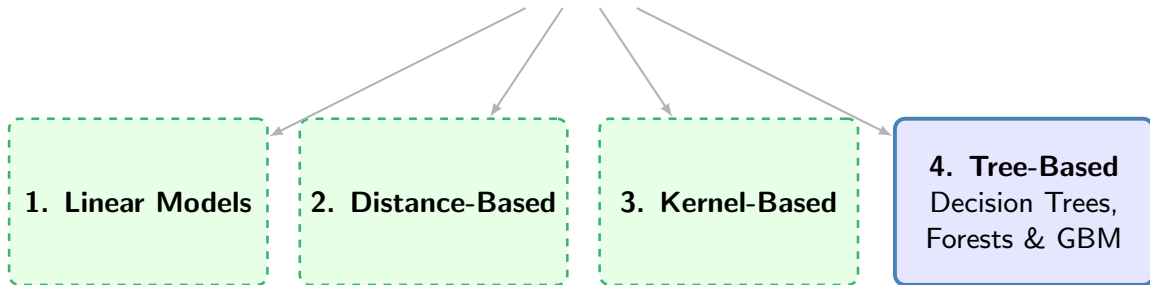
Advantages

- ▶ **High Dimensionality:** Works very well when *features* $\gg$ *samples*.
- ▶ **Powerful:** Different kernels allow for modeling complex non-linear relationships.

Disadvantages

- ▶ **Noise Sensitivity:** Sensitive to noise and outliers (if C is large).
- ▶ **Scalability:** Slow for large datasets ($> 10k$ samples).
- ▶ **Preprocessing:** Requires feature scaling.

Machine Learning Models (f_{θ})

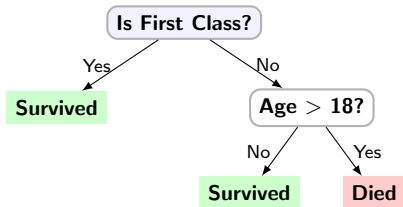


↑ We Are Here

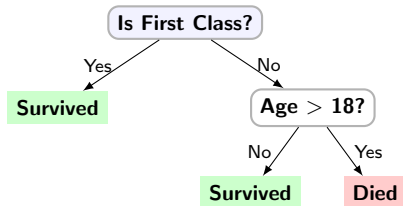
Intuition: We split data by asking a sequence of Yes/No questions to separate classes.

Example: Titanic Survival

- ▶ **Goal:** Predict who survived.
- ▶ **Root Question:** "Was the passenger in First Class?" (Best separator).
- ▶ **Next Question:** If not, "Were they a child?"



Intuition: We split data by asking a sequence of Yes/No questions to separate classes.



Example: Titanic Survival

- ▶ **Goal:** Predict who survived.
- ▶ **Root Question:** "Was the passenger in First Class?" (Best separator).
- ▶ **Next Question:** If not, "Were they a child?"

The Core Question

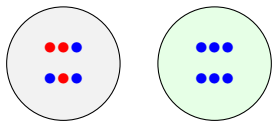
We have many features (Age, Ticket, Gender...).

How do we decide which question to ask first?

How to Choose the Best Split? (Purity)

We want questions that lead to **"Pure" nodes** (containing only one class). We measure "disorder" using **Impurity Metrics**.

Visualizing Impurity



High Impurity
(Mixed)

Zero Impurity
(Pure)

The Algorithm's Goal: Pick the split that maximizes the drop in Impurity (Information Gain).

The "Impurity Metrics":

1. Gini Impurity (Default):

$$G = 1 - \sum p_i^2$$

- $G = 0.5$: Max chaos (50/50).
- $G = 0.0$: Pure (One class).

2. Entropy (Information):

$$H = - \sum p_i \log_2(p_i)$$

What is p_i ?

p_i is the **proportion** of samples belonging to class i in the current node.

Wait... Where is Gradient Descent?

Q: Do Decision Trees use Gradient Descent to find the best split?

Q: Do Decision Trees use Gradient Descent to find the best split?

The Answer: NO!

Standard Decision Trees are **Non-Differentiable**. You cannot calculate a gradient ∇J because splits are discrete "hard" choices (Left vs. Right).

Comparison of Learning Algorithms:

Non-Trees Models

Method: Gradient Descent

- ▶ Loss surface is smooth (convex).
- ▶ We slide down the curve.
- ▶ Update weights $\mathbf{w}$ iteratively.

Decision Trees

Method: Greedy Search

- ▶ Try **every possible split** for every feature.
- ▶ Pick the one with max Information Gain.
- ▶ **Repeat recursively until:**
 - The node is **Pure** (100% one class).
 - Or we hit a **Stop Condition** (e.g., Max Depth).

The Advantages

- ▶ **Simple:** Easy to understand.
- ▶ **Interpretable:** Easy to explain to non-experts ("If $X > 5$, then Y ").
- ▶ **No Preprocessing Required:** No feature scaling or encoding required.

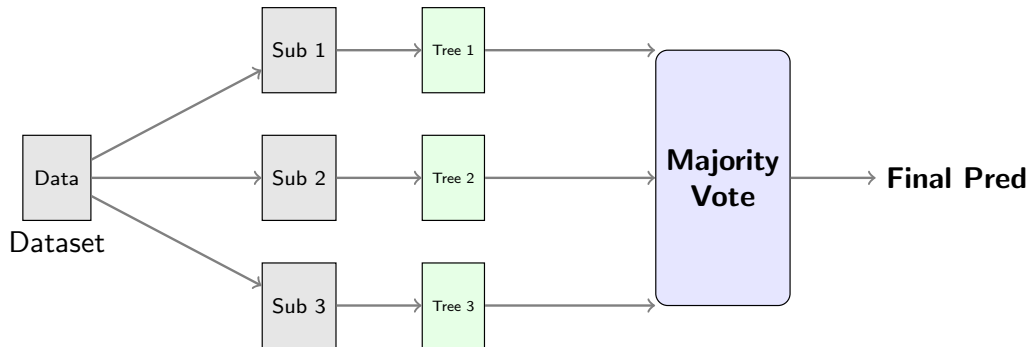
The Disadvantages

- ▶ **High Variance:** A small change in data = completely different tree.
- ▶ **Overfitting:** Not setting the stop conditions (e.g., Max Depth) guarantees overfitting.

Problem: Single trees are unstable (High Variance).

Problem: Single trees are unstable (High Variance).

Solution: Train many trees on random subsets and average them (**Bagging**).



"Many uncorrelated weak models → One strong model."

Advantages

- ▶ **Robust:** Very hard to overfit.
- ▶ **High Performance:** Very good baseline.
- ▶ **Low Hyperparameters Sensitivity:** Works well with default hyperparameters.
- ▶ **Interpretability:** Tells you which features matter most (Feature Importance).

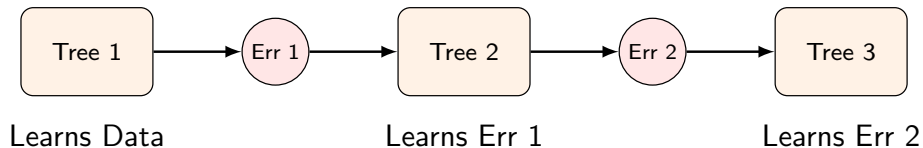
Disadvantages

- ▶ **Slow Inference:** Must run data through every single tree.
- ▶ **Memory Intensive:** Stores all trees in RAM.

Problem: Random Forest just averages opinions. It doesn't learn from mistakes.

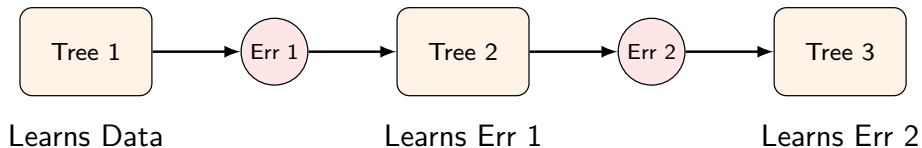
Problem: Random Forest just averages opinions. It doesn't learn from mistakes.

Solution: Train trees **sequentially**. Each tree fixes the errors of the previous one.



Problem: Random Forest just averages opinions. It doesn't learn from mistakes.

Solution: Train trees **sequentially**. Each tree fixes the errors of the previous one.



Common Implementations:

- ▶ **XGBoost:** Best Default
- ▶ **LightGBM:** Lightweight
- ▶ **CatBoost:** Requires minimal hyperparamters tuning.

Why is it called *Gradient* Boosting?

We are not just "fixing errors." We are performing **Gradient Descent**, but in a different way.

1. Gradient Descent

In Linear Regression, we update **parameters** (w) to minimize Loss.

$$\mathbf{w}_{new} = \mathbf{w}_{old} \underbrace{-\alpha \cdot \nabla \text{Loss}}_{\text{Step down the hill}}$$

2. Gradient Boosting

Here, we update the **function** (F) by adding new trees.

$$F_{new}(x) = F_{old}(x) + \underbrace{\alpha \cdot \text{Tree}(x)}_{\text{Step down the hill}}$$

- ▶ If we use **Squared Error** Loss: $L = \frac{1}{2}(y - \hat{y})^2$
- ▶ The Gradient (derivative) is: $\nabla L = -(y - \hat{y})$
- ▶ Therefore: **The Residual (which we fit trees on) ($y - \hat{y}$) is effectively the Negative Gradient!**
- ▶ Final Model = Base Tree + $\sum$ (Error-Correcting Trees).

Advantages

- ▶ **SOTA Performance:** Best accuracy for tabular data.
- ▶ **Flexibility:** Customizable Loss functions.
- ▶ **No Preprocessing Required:** Handles Missing Values natively. No need for scaling.

Disadvantages

- ▶ **Sensitive:** Harder to tune (learning rate, depth).
- ▶ **Overfitting:** Easy to overfit if not careful (too many trees).
- ▶ **Serial:** Slow training (hard to parallelize).
- ▶ **Data Hungry:** Can overfit easily if data is small.

"Is there one Master Algorithm to rule them all?"

The Theorem (Wolpert, 1996)

Averaged over all possible data generating distributions, every classification algorithm has the same error rate when classifying previously unobserved points.

$$\mathbb{E}[\text{Error}(\text{Model}_A)] = \mathbb{E}[\text{Error}(\text{Model}_B)]$$

In Plain English:

- ▶ No model works best for every problem.
- ▶ A model that is great at Image Recognition might be terrible at Spam Classification.

The Implication for us

Since we cannot mathematically prove which model will work best beforehand:

1. We must make **assumptions** about our data (e.g., "it looks linear").
2. We must **experiment** with multiple models from our toolbox.

- ▶ Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*
- ▶ Andrew Ng, *Machine Learning Specialization* (Coursera/DeepLearning.AI)

Slides contributed by Mohamed Eltayeb